ASSIGNMENT - 11.3

NAME : K.ABHINAY

ROLL NO. : 2303A51899

BATCH NO. : 09

Lab 11: Data Structures with AI Implementing Fundamental Data Structures using AI Assistance
Lab Objectives:
By the end of this lab, students will be able to:
Week6 -
Wednesday
• Design and implement fundamental data structures in Python using AI assistance.
• Effectively prompt AI tools (e.g., GitHub Copilot) for code generation, optimization, and documentation.
• Understand and compare core data structures: Arrays, Linked Lists, Stacks, Queues, Priority Queues, Trees, and Graphs.
• Improve code readability, efficiency, and maintainability using AI-generated suggestions.
Learning Outcomes
After completing this lab, students will be able to:
• Apply appropriate data structures to solve real-world problems.
• Analyze time and space complexity of different data structure operations.
• Use AI tools responsibly to assist (not replace) logical thinking and problem-solving.
• Validate, test, and refine AI-generated code.

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario
SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.
Tasks
1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:
o Add a contact
o Search for a contact
o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:
o Insertion efficiency
o Deletion efficiency
Expected Outcome
• Two working implementations (array-based and linked-list-based).
• A brief comparison explaining performance differences.

CODE:

```
# Lab 11: Data Structures with AI Assistance
# TASK 1: Smart Contact Manager (Arrays & Linked Lists)

# Array-based Contact Manager
class ArrayContactManager:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        """Add a contact to the array."""
        self.contacts.append({'name': name, 'phone': phone})
        print(f"[Array] Contact '{name}' added.")

    def search_contact(self, name):
        """Search for a contact by name (linear search)."""
        for contact in self.contacts:
            if contact['name'].lower() == name.lower():
```

```
                print(f"[Array] Found: {contact}")
                return contact
        print(f"[Array] Contact '{name}' not found.")
        return None

    def delete_contact(self, name):
        """Delete a contact by name."""
        for i, contact in enumerate(self.contacts):
            if contact['name'].lower() == name.lower():
                removed = self.contacts.pop(i)
                print(f"[Array] Deleted: {removed}")
                return
        print(f"[Array] Contact '{name}' not found.")

    def display_all(self):
        print("\n[Array] All Contacts:")
        if not self.contacts:
            print("  (empty)")
        for c in self.contacts:
            print(f"  {c['name']} : {c['phone']}")


# Linked List-based Contact Manager
class ContactNode:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None


class LinkedListContactManager:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        """Add a contact at the end of the linked list."""
        new_node = ContactNode(name, phone)
        if not self.head:
            self.head = new_node
        else:
```

```
            curr = self.head
            while curr.next:
                curr = curr.next
            curr.next = new_node
        print(f"[LinkedList] Contact '{name}' added.")

    def search_contact(self, name):
        """Traverse and search for a contact."""
        curr = self.head
        while curr:
            if curr.name.lower() == name.lower():
                print(f"[LinkedList] Found: {curr.name} : {curr.phone}")
                return curr
            curr = curr.next
        print(f"[LinkedList] Contact '{name}' not found.")
        return None

    def delete_contact(self, name):
        """Delete a contact node by name."""
        curr = self.head
        prev = None
        while curr:
            if curr.name.lower() == name.lower():
                if prev:
                    prev.next = curr.next
                else:
                    self.head = curr.next
                print(f"[LinkedList] Deleted: {curr.name}")
                return
            prev = curr
            curr = curr.next
        print(f"[LinkedList] Contact '{name}' not found.")

    def display_all(self):
        print("\n[LinkedList] All Contacts:")
        curr = self.head
        if not curr:
            print("  (empty)")
        while curr:
            print(f"  {curr.name} : {curr.phone}")
```

```
            curr = curr.next
```

OUTPUT:

```
TERMINAL                                                                    Python
PS C:\Users\abhin> & C:/Users/abhin/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/abhin/OneDrive/Desktop/Assignment11.py
  ============================================================
  TASK 1: Smart Contact Manager
  ============================================================

  --- Array-based ---
  [Array] Contact 'Ravi' added.
  [Array] Contact 'Sneha' added.
  [Array] Contact 'Arjun' added.

  [Array] All Contacts:
    Ravi : 9876543210
    Sneha : 9123456789
    Arjun : 9000011111
  [Array] Found: {'name': 'Sneha', 'phone': '9123456789'}
  [Array] Deleted: {'name': 'Ravi', 'phone': '9876543210'}

  [Array] All Contacts:
    Sneha : 9123456789
    Arjun : 9000011111

  --- LinkedList-based ---
  [LinkedList] Contact 'Ravi' added.
  [LinkedList] Contact 'Sneha' added.
  [LinkedList] Contact 'Arjun' added.

  [LinkedList] All Contacts:
    Ravi : 9876543210
    Sneha : 9123456789
    Arjun : 9000011111
  [LinkedList] Found: Sneha : 9123456789
  [LinkedList] Deleted: Ravi

  [LinkedList] All Contacts:
    Sneha : 9123456789
    Arjun : 9000011111

  --- Comparison ---
    Insertion: Array O(1) append | LinkedList O(n) traverse to end
    Deletion:  Array O(n) shift  | LinkedList O(n) traverse, O(1) pointer update
    Search:    Both O(n) linear search
```

Task 2:

Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:

o enqueue() method

o dequeue() method

4. Test the system with a mix of student and faculty requests.

Expected Outcome

• Working queue and priority queue implementations.

• Correct prioritization of faculty requests.

CODE:

```
# TASK 2: Library Book Search System (Queue & Priority Queue)
from collections import deque
import heapq

class BookRequestQueue:
    """Simple FIFO Queue for book borrow requests."""
    def __init__(self):
        self.queue = deque()

    def enqueue(self, requester, book):
        """Add a book request to the queue."""
        self.queue.append({'requester': requester, 'book': book})
        print(f"[Queue] Request added: {requester} -> '{book}'")

    def dequeue(self):
        """Process the next request (FIFO)."""
        if self.queue:
            req = self.queue.popleft()
            print(f"[Queue] Processing: {req['requester']} ->
'{req['book']}'")
            return req
        print("[Queue] No pending requests.")
        return None

    def display(self):
        print("\n[Queue] Pending Requests:")
        if not self.queue:
            print("  (empty)")
        for r in self.queue:
            print(f"  {r['requester']} -> '{r['book']}'")


class PriorityBookQueue:
    """
    Priority Queue: Faculty (priority=1) > Student (priority=2).
    Uses heapq; lower number = higher priority.
    """
```

```
    def __init__(self):
        self.heap = []
        self.counter = 0

    def enqueue(self, requester, book, role='student'):
        """Add a request; faculty gets higher priority."""
        priority = 1 if role.lower() == 'faculty' else 2
        heapq.heappush(self.heap, (priority, self.counter, requester,
book))
        self.counter += 1
        print(f"[PriorityQueue] Added ({role}): {requester} -> '{book}'")

    def dequeue(self):
        """Process highest-priority request."""
        if self.heap:
            priority, _, requester, book = heapq.heappop(self.heap)
            role = 'Faculty' if priority == 1 else 'Student'
            print(f"[PriorityQueue] Processing ({role}): {requester} ->
'{book}'")
            return (requester, book)
        print("[PriorityQueue] No pending requests.")
        return None

    def display(self):
        print("\n[PriorityQueue] Pending Requests (sorted by priority):")
        if not self.heap:
            print("  (empty)")
        for item in sorted(self.heap):
            role = 'Faculty' if item[0] == 1 else 'Student'
            print(f"  [{role}] {item[2]} -> '{item[3]}'")
```

OUTPUT:

```
TERMINAL
PS C:\Users\abhin> & C:/Users/abhin/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/abhin/OneDrive/Desktop/Assignment11.py

============================================================
TASK 2: Library Book Search System
============================================================

--- Simple Queue ---
[Queue] Request added: Alice (Student) -> 'Data Structures'
[Queue] Request added: Dr. Mehta (Faculty) -> 'Algorithms'
[Queue] Request added: Bob (Student) -> 'DBMS'

[Queue] Pending Requests:
  Alice (Student) -> 'Data Structures'
  Dr. Mehta (Faculty) -> 'Algorithms'
  Bob (Student) -> 'DBMS'
[Queue] Processing: Alice (Student) -> 'Data Structures'
[Queue] Processing: Dr. Mehta (Faculty) -> 'Algorithms'

--- Priority Queue ---
[PriorityQueue] Added (student): Alice -> 'Data Structures'
[PriorityQueue] Added (faculty): Dr. Mehta -> 'Algorithms'
[PriorityQueue] Added (student): Bob -> 'DBMS'
[PriorityQueue] Added (faculty): Prof. Rao -> 'ML Basics'

[PriorityQueue] Pending Requests (sorted by priority):
  [Faculty] Dr. Mehta -> 'Algorithms'
  [Faculty] Prof. Rao -> 'ML Basics'
  [Student] Alice -> 'Data Structures'
  [Student] Bob -> 'DBMS'

Processing all:
[PriorityQueue] Processing (Faculty): Dr. Mehta -> 'Algorithms'
[PriorityQueue] Processing (Faculty): Prof. Rao -> 'ML Basics'
[PriorityQueue] Processing (Student): Alice -> 'Data Structures'
[PriorityQueue] Processing (Student): Bob -> 'DBMS'
```

Task 3:

Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.
2. Provide the following operations:
o push(ticket)
o pop()
o peek()
3. Simulate at least five tickets being raised and resolved.
4. Use GitHub Copilot to suggest additional stack operations such as:
o Checking whether the stack is empty
o Checking whether the stack is full (if applicable)

Expected Outcome

• Functional stack-based ticket management system.
• Clear demonstration of LIFO behavior.

CODE:

```
# TASK 3: Emergency Help Desk (Stack Implementation)
```

```python
class HelpDeskStack:
    """Stack-based IT Help Desk ticket manager (LIFO)."""

    def __init__(self, max_size=10):
        self.stack = []
        self.max_size = max_size

    def push(self, ticket):
        """Push a new ticket onto the stack."""
        if self.is_full():
            print(f"[Stack] Stack full! Cannot add ticket: '{ticket}'")
        else:
            self.stack.append(ticket)
            print(f"[Stack] Ticket pushed: '{ticket}'")

    def pop(self):
        """Resolve (remove) the most recent ticket."""
        if self.is_empty():
            print("[Stack] No tickets to resolve.")
            return None
        resolved = self.stack.pop()
        print(f"[Stack] Resolved ticket: '{resolved}'")
        return resolved

    def peek(self):
        """View the most recent ticket without removing it."""
        if self.is_empty():
            print("[Stack] Stack is empty.")
            return None
        print(f"[Stack] Current top ticket: '{self.stack[-1]}'")
        return self.stack[-1]

    def is_empty(self):
        return len(self.stack) == 0

    def is_full(self):
        return len(self.stack) >= self.max_size
```

```python
    def display(self):
        print("\n[Stack] All Tickets (top → bottom):")
        if self.is_empty():
            print("  (empty)")
        for ticket in reversed(self.stack):
            print(f"  - {ticket}")
```

OUTPUT:

```
PS C:\Users\abhin> & C:/Users/abhin/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/abhin/OneDrive/Desktop/Assignment11.py
[PriorityQueue] Processing (Student): Bob -> 'DBMS'

============================================================
TASK 3: Emergency Help Desk (Stack)
============================================================
[Stack] Ticket pushed: 'Ticket-1: WiFi not working - Ravi'
[Stack] Ticket pushed: 'Ticket-2: Printer jammed - Sneha'
[Stack] Ticket pushed: 'Ticket-3: Login issue - Arjun'
[Stack] Ticket pushed: 'Ticket-4: Software crash - Priya'
[Stack] Ticket pushed: 'Ticket-5: VPN error - Kiran'

[Stack] All Tickets (top → bottom):
  - Ticket-5: VPN error - Kiran
  - Ticket-4: Software crash - Priya
  - Ticket-3: Login issue - Arjun
  - Ticket-2: Printer jammed - Sneha
  - Ticket-1: WiFi not working - Ravi
[Stack] Current top ticket: 'Ticket-5: VPN error - Kiran'

[Stack] Resolved ticket: 'Ticket-5: VPN error - Kiran'
[Stack] Resolved ticket: 'Ticket-4: Software crash - Priya'
[Stack] Current top ticket: 'Ticket-3: Login issue - Arjun'
Stack empty? False

[Stack] All Tickets (top → bottom):
  - Ticket-3: Login issue - Arjun
  - Ticket-2: Printer jammed - Sneha
  - Ticket-1: WiFi not working - Ravi
```

Task 4:

Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

• Insert

• Search

• Delete

Starter Code

class HashTable:

pass

Expected Outcome

• Collision handling using chaining

• Well-commented methods

CODE:

```
# TASK 4: Hash Table with Chaining (Collision Handling)

class HashTable:
    """
    Hash Table using separate chaining for collision handling.
    Each bucket holds a list of (key, value) pairs.
    """

    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(self.size)]  # Array of buckets

    def _hash(self, key):
        """Compute hash index for a given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert or update a key-value pair."""
        index = self._hash(key)
        bucket = self.table[index]

        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                print(f"[HashTable] Updated key '{key}' -> '{value}'")
                return
        bucket.append((key, value))
        print(f"[HashTable] Inserted key '{key}' -> '{value}'")

    def search(self, key):
        """Search for a value by key."""
        index = self._hash(key)
        bucket = self.table[index]
        for k, v in bucket:
            if k == key:
                print(f"[HashTable] Found '{key}' -> '{v}'")
```

```python
                return v
        print(f"[HashTable] Key '{key}' not found.")
        return None

    def delete(self, key):
        """Delete a key-value pair."""
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                del bucket[i]
                print(f"[HashTable] Deleted key '{key}'")
                return
        print(f"[HashTable] Key '{key}' not found.")

    def display(self):
        """Display all buckets."""
        print("\n[HashTable] Contents:")
        for i, bucket in enumerate(self.table):
            if bucket:
                print(f"  Bucket {i}: {bucket}")
```

OUTPUT:

```
PS C:\Users\abhin> & C:/Users/abhin/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/abhin/OneDrive/Desktop/Assignment11.py

============================================================
TASK 4: Hash Table
============================================================
[HashTable] Inserted key 'S001' -> 'Ravi Kumar'
[HashTable] Inserted key 'S002' -> 'Sneha Rao'
[HashTable] Inserted key 'S003' -> 'Arjun Reddy'
[HashTable] Updated key 'S001' -> 'Ravi K. (Updated)'

[HashTable] Contents:
  Bucket 0: [('S001', 'Ravi K. (Updated)')]
  Bucket 1: [('S002', 'Sneha Rao')]
  Bucket 4: [('S003', 'Arjun Reddy')]
[HashTable] Found 'S002' -> 'Sneha Rao'
[HashTable] Key 'S999' not found.
[HashTable] Deleted key 'S003'

[HashTable] Contents:
  Bucket 0: [('S001', 'Ravi K. (Updated)')]
  Bucket 1: [('S002', 'Sneha Rao')]
```

Task 5:

Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

• Student Attendance Tracking

• Event Registration System
• Library Book Borrowing
• Bus Scheduling System
• Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

• Mapping table: Feature → Data Structure → Justification
• One fully working Python implementation

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

CODE:

```
# TASK 5: Real-Time Application - Cafeteria Order Queue
# Feature → Data Structure Mapping:
# Implemented Feature: Cafeteria Order Queue

class CafeteriaOrderQueue:
    """
    FIFO Queue for managing cafeteria orders.
    Each order has a student name and food item.
    """

    def __init__(self):
        self.queue = deque()
        self.order_id = 1

    def place_order(self, student, item):
        """Student places a food order."""
        order = {'id': self.order_id, 'student': student, 'item': item}
        self.queue.append(order)
        print(f"[Cafeteria] Order #{self.order_id} placed by {student}:
'{item}'")
        self.order_id += 1
```

```
    def serve_order(self):
        """Serve the next order in the queue."""
        if self.queue:
            order = self.queue.popleft()
            print(f"[Cafeteria] Serving Order #{order['id']}:
{order['student']} -> '{order['item']}'")
            return order
        print("[Cafeteria] No pending orders.")
        return None

    def display_orders(self):
        print("\n[Cafeteria] Pending Orders:")
        if not self.queue:
            print("  (empty)")
        for order in self.queue:
            print(f"  #{order['id']} | {order['student']} |
'{order['item']}'")


# MAIN - Demo / Test all Tasks


if __name__ == "__main__":

    print("=" * 60)
    print("TASK 1: Smart Contact Manager")
    print("=" * 60)

    print("\n--- Array-based ---")
    acm = ArrayContactManager()
    acm.add_contact("Ravi", "9876543210")
    acm.add_contact("Sneha", "9123456789")
    acm.add_contact("Arjun", "9000011111")
    acm.display_all()
    acm.search_contact("Sneha")
    acm.delete_contact("Ravi")
    acm.display_all()

    print("\n--- LinkedList-based ---")
    llcm = LinkedListContactManager()
```

```
    llcm.add_contact("Ravi", "9876543210")
    llcm.add_contact("Sneha", "9123456789")
    llcm.add_contact("Arjun", "9000011111")
    llcm.display_all()
    llcm.search_contact("Sneha")
    llcm.delete_contact("Ravi")
    llcm.display_all()

    print("\n--- Comparison ---")
    print("  Insertion: Array O(1) append | LinkedList O(n) traverse to
end")
    print("  Deletion:  Array O(n) shift  | LinkedList O(n) traverse, O(1)
pointer update")
    print("  Search:    Both O(n) linear search")

    print("\n" + "=" * 60)
    print("TASK 2: Library Book Search System")
    print("=" * 60)

    print("\n--- Simple Queue ---")
    bq = BookRequestQueue()
    bq.enqueue("Alice (Student)", "Data Structures")
    bq.enqueue("Dr. Mehta (Faculty)", "Algorithms")
    bq.enqueue("Bob (Student)", "DBMS")
    bq.display()
    bq.dequeue()
    bq.dequeue()

    print("\n--- Priority Queue ---")
    pq = PriorityBookQueue()
    pq.enqueue("Alice", "Data Structures", role="student")
    pq.enqueue("Dr. Mehta", "Algorithms", role="faculty")
    pq.enqueue("Bob", "DBMS", role="student")
    pq.enqueue("Prof. Rao", "ML Basics", role="faculty")
    pq.display()
    print("\nProcessing all:")
    pq.dequeue()
    pq.dequeue()
    pq.dequeue()
    pq.dequeue()
```

```python
print("\n" + "=" * 60)
print("TASK 3: Emergency Help Desk (Stack)")
print("=" * 60)

hd = HelpDeskStack()
hd.push("Ticket-1: WiFi not working - Ravi")
hd.push("Ticket-2: Printer jammed - Sneha")
hd.push("Ticket-3: Login issue - Arjun")
hd.push("Ticket-4: Software crash - Priya")
hd.push("Ticket-5: VPN error - Kiran")
hd.display()
hd.peek()
print()
hd.pop()
hd.pop()
hd.peek()
print(f"Stack empty? {hd.is_empty()}")
hd.display()

print("\n" + "=" * 60)
print("TASK 4: Hash Table")
print("=" * 60)

ht = HashTable(size=7)
ht.insert("S001", "Ravi Kumar")
ht.insert("S002", "Sneha Rao")
ht.insert("S003", "Arjun Reddy")
ht.insert("S001", "Ravi K. (Updated)")
ht.display()
ht.search("S002")
ht.search("S999")
ht.delete("S003")
ht.display()

print("\n" + "=" * 60)
print("TASK 5: Cafeteria Order Queue")
print("=" * 60)

cafe = CafeteriaOrderQueue()
```

```python
    cafe.place_order("Ravi", "Veg Biryani")
    cafe.place_order("Sneha", "Masala Dosa")
    cafe.place_order("Arjun", "Cold Coffee")
    cafe.place_order("Priya", "Paneer Wrap")
    cafe.display_orders()
    print("\nServing orders:")
    cafe.serve_order()
    cafe.serve_order()
    cafe.display_orders()
```

OUTPUT:

```
TERMINAL                                                                    Python
PS C:\Users\abhin> & C:/Users/abhin/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/abhin/OneDrive/Desktop/Assignment11.py

============================================================
TASK 5: Cafeteria Order Queue
============================================================
[Cafeteria] Order #1 placed by Ravi: 'Veg Biryani'
[Cafeteria] Order #2 placed by Sneha: 'Masala Dosa'
[Cafeteria] Order #3 placed by Arjun: 'Cold Coffee'
[Cafeteria] Order #4 placed by Priya: 'Paneer Wrap'

[Cafeteria] Pending Orders:
  #1 | Ravi | 'Veg Biryani'
  #2 | Sneha | 'Masala Dosa'
  #3 | Arjun | 'Cold Coffee'
  #4 | Priya | 'Paneer Wrap'

Serving orders:
[Cafeteria] Serving Order #1: Ravi -> 'Veg Biryani'
[Cafeteria] Serving Order #2: Sneha -> 'Masala Dosa'

[Cafeteria] Pending Orders:
  #3 | Arjun | 'Cold Coffee'
  #4 | Priya | 'Paneer Wrap'
  #3 | Arjun | 'Cold Coffee'
  #4 | Priya | 'Paneer Wrap'
PS C:\Users\abhin>
```